

Práctica de Eliminación Lógica y Actualizaciones

Objetivo

Partiendo del proyecto "**Práctica de Validaciones con Express Validator**", se deberá incorporar eliminación lógica en al menos un modelo, y agregar validaciones para las rutas de actualización utilizando **.optional()** y **matchedData()**.

Criterios de Evaluación

1. Presentación del código

- El código debe estar limpio, ordenado y bien indentado.
- Uso obligatorio de try-catch en todos los controladores para un manejo adecuado de errores.
- Organización correcta del proyecto en carpetas:
 - src/config → configuración (conexión a la base de datos).
 - src/models → definición de modelos Sequelize.
 - src/routes → definición de rutas.
 - src/controllers → controladores.
 - src/middlewares → middlewares.
- Uso exclusivo de ESModules (import / export).
- El código debe ser funcional, modularizado y sin errores de ejecución.

2. Validaciones y lógica

- Validaciones correctamente implementadas con **express-validator**.
- Respuestas claras y códigos HTTP apropiados en cada operación:
 - 201 Created → al crear un recurso.
 - 200 OK → en consultas y actualizaciones exitosas.
 - 400 Bad Request → errores de validación.
 - 404 Not Found → recurso inexistente.
 - 500 Internal Server Error → errores inesperados del servidor.
- Mensajes de éxito o error claros al realizar operaciones CRUD.

Entrega mediante Git y GitHub – Uso de ramas y commits

Teniendo en cuenta que al finalizar la práctica anterior las ramas main y develop quedaron sincronizadas, ahora se debe seguir el siguiente flujo:

- Crear una rama a partir de **develop** llamada **eliminacion-logica**.
- Realizar todo el desarrollo en la rama **eliminacion-logica**.
- Al finalizar el trabajo:
 - Hacer un merge limpio de **eliminacion-logica** hacia develop, sin conflictos.
 - Luego hacer un merge limpio de develop hacia main, para que ambas ramas queden sincronizadas nuevamente.
- Durante el desarrollo en la rama **eliminacion-logica**, se deben realizar **al menos 3 commits** con mensajes claros y descriptivos.

Consignas

1. Rutas de actualización y eliminación

Para poder aplicar validaciones y eliminación lógica, primero es necesario contar con las rutas correspondientes en todos los modelos. Se deben implementar las rutas faltantes para cada recurso existente en el proyecto:

- PUT /api/{nombre}/:id → actualizar un registro por ID.
- DELETE /api/{nombre}/:id → eliminar un registro por ID.

Reemplazar {nombre} por el nombre del recurso en plural y minúsculas (**por ejemplo: users, tasks, profiles, roles**).

2. Eliminación lógica

Se debe implementar eliminación lógica en al menos uno de los modelos existentes. La eliminación lógica consiste en no borrar físicamente el registro de la base de datos, sino marcarlo como eliminado mediante un campo adicional, de modo que pueda ser excluido de las consultas sin perder el historial de datos.

Para implementarla se debe utilizar la opción paranoid de Sequelize, que automatiza este comportamiento. Al activarla en un modelo, Sequelize agrega automáticamente el campo **deletedAt** y, en lugar de eliminar el registro, registra la fecha y hora de eliminación en ese campo. Las consultas estándar excluyen automáticamente los registros con **deletedAt** distinto de null.

Para implementarla se debe:

- Activar **paranoid: true** en las opciones del modelo elegido.
- Asegurarse de que el modelo también tenga **timestamps: true**, ya que paranoid depende de esta opción.
- Verificar que el controlador de eliminación utilice el método **.destroy()** normalmente, ya que Sequelize se encarga de realizar la eliminación lógica de forma transparente.

3. Validaciones para rutas de actualización

Las rutas de actualización (**PUT**) tienen una lógica de validación diferente a las de creación: no todos los campos son obligatorios, ya que el usuario puede querer actualizar solo algunos de ellos.

Para este tipo de rutas se debe utilizar **.optional()** en las validaciones, de manera que un campo solo sea validado si está presente en el cuerpo de la solicitud, pero no sea rechazado si se omite.

Además, en el controlador se debe utilizar **matchedData()** para obtener únicamente los campos que fueron enviados y validados, evitando pasar al modelo datos no esperados o no validados provenientes del **req.body**.

Se deben agregar validaciones con estas características en todos los controladores de actualización que aún no las tengan.